

## 8.2 IEEE Operation

### 8.2.1 Hardware Configuration

The interface comes with a 24-pin female AMPHENOL connector that complies with the IEEE-488 standard. If your controller cabling is of the IEC-625 type, connection can be made using the IEEE to IEC adapter PM 9483/50 (to be ordered separately, not included as accessory).

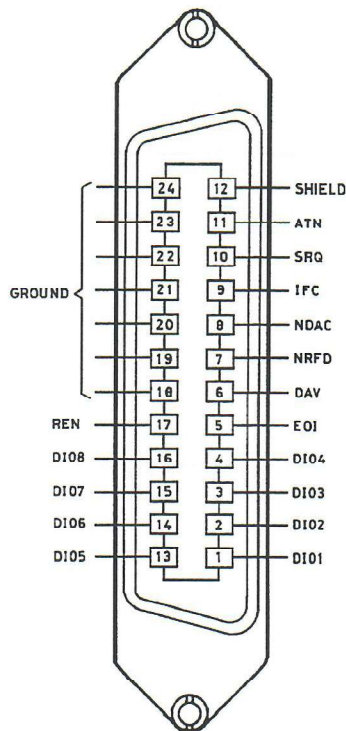


Fig. 8-1 Connector pin-out, IEEE

The hardware configurationals include:

- Device address
- Interface mode
- Shielding

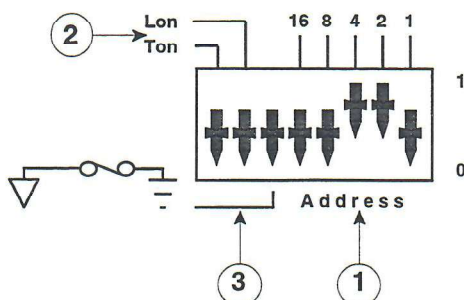


Fig. 8-2 Hardware programming, SW1

1:

The device address is set using the five decimal weighted switches. All address settings except "31" and the "[controllers address]" are acceptable (standard factory setting is "6").

Example:

|   | 16 | 8 | 4 | 2 | 1 |
|---|----|---|---|---|---|
| 0 | X  | X |   |   | X |
| 1 |    |   | X | X |   |

Address = 4 + 2 = 6

2:

#### Mode Switches

| Lon | Ton | Mode                                           |
|-----|-----|------------------------------------------------|
| 0   | 0   | Addressable mode<br>(standard factory setting) |
| 0   | 1   | Talk only                                      |
| 1   | 0   | Listen only                                    |

3:

Logic 0 may be disconnected from the shielding in order to eliminate unwanted currents.

| Switch position | Shield condition                                  |
|-----------------|---------------------------------------------------|
| 0               | Logic "0" connected<br>(standard factory setting) |
| 1               | Logic "0" disconnected                            |

### 8.2.2 Interface programming

The IEEE remote interface can via the controller be configured with regard to:

- Record separator
- Service request mask

Furthermore, it can provide 'device indication'. The programming of the interface must follow the syntax illustrated on page 8-1.

**Note:** When programming the interface, before every header/body combination, the listen address of the interface must be send (MLA).

### 1. Separator programming

The interface can be programmed to accept single and double separators. At power on the single separator "LF" is default setting (ISO code 10). When the interface sends data to the controller, the data stream is concluded with the programmed separator(s), and the EOI line is activated concurrent with the transmission of the (last) separator.

#### Definition of record separator(s).

| HEADER | DATA                                                                                                                                                                                                                             | SUFFIX | RETURNS |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|---------|
| SPR    | n1 (,n2)<br>Separator 1 : n1<br>Separator 2 : n2<br>n1 and n2 are the decimal equivalent (ISO code) of the desired separator(s).<br>ESC (ISO 27) is not allowed as separator.<br>Default separator at power on is "LF" (ISO 10). | none   |         |

Table 8-1 Programming, separator(s)

#### Example:

- a) "SPR 03" defines "ETX" as input/output separator.
- b) "SPR 13, 10" defines the sequence "CR LF" as input/output separator.

### 2. Service request mask

The interface can be programmed to request service for one or more of the conditions that are reflected in the status byte.

| Bit | Condition |
|-----|-----------|
| 7   | EXT       |
| 6   | RQS       |
| 5   | AB        |
| 4   | BSY       |
| 3   | EF3       |
| 2   | EF2       |
| 1   | EF1       |
| 0   | EF0       |

Table 8-2 Status byte

#### EXT:

not used

#### RQS:

service request

#### AB:

Abnormal condition

| AB |                                                     |
|----|-----------------------------------------------------|
| 0  | Normal (no error)                                   |
| 1  | Abnormal (error condition as specified by EF0-EF3)) |

**BSY:**

Busy

**EF0-EF3:**

Message flags

| AB | EF3-EF0 | Dec. | Message                  |
|----|---------|------|--------------------------|
| 0  | 0001    | 1    | Message string available |
| 1  | 1100    | 12   | Number error             |
| 1  | 1101    | 13   | Execute error            |
| 1  | 1110    | 14   | Head/suffix error        |
| 1  | 1111    | 15   | Syntax error             |

Table 8-3 Message flags EF0-EF3

At power on all service requests are masked (disabled). Thus, if an error condition occurs, no service request will be generated. The instrument status can, however, always be checked by reading the status byte via a serial poll. The service request facility can be enabled for the following qualifying conditions:

| Qualifying condition                   | Decimal equivalent |
|----------------------------------------|--------------------|
| EF0 changed from 0 to 1 and AB=0 (DAV) | 1                  |
| EF1 changed and AB=0 (not used)        | 2                  |
| EF2 changed and AB=0 (not used)        | 4                  |
| EF3 changed and AB=0 (not used)        | 8                  |
| EF0 changed from 0 to 1 and AB=1       | 16                 |
| EF1 changed from 0 to 1 and AB=1       | 32                 |
| EF2 changed from 0 to 1 and AB=1       | 64                 |
| EF3 changed from 0 to 1 and AB=1       | 128                |
| BUSY changed from 1 to 0               | 256                |

Table 8-4 Qualifying conditions

At the generation of a service request the RQS bit will be set and the SRQ line activated. RQS and SRQ are reset again, when the controller polls the status byte. To specify one or more qualifying conditions, the decimal equivalent of the condition(s) must be send to the interface using the MSR command ("MSR 0" masks (disables) all interrupts):

**Definition of service request mask**

| HEADER | DATA                      | SUFFIX | RETURNS |
|--------|---------------------------|--------|---------|
| MSR    | n<br>Mask : n = [0 - 511] | none   |         |

Table 8-5 Programming, service request mask

**Example 1:**

The interface shall request service for 'data available' only. The mask bit for 'data available' has the decimal value "1". The resulting command is "MSR 1".

**Example 2:**

The interface shall request service for 'data available' and for all the possible error conditions reflected by the error flags EF3-EF0. The EF3 bit be active in all four possible error conditions. The decimal value for the desired qualifying conditions is 129 (128+1) as shown in table 6-6. The resulting command is "MSR 129".

**Note:** The status byte can not simultaneously indicate conditions related to AB=0 (normal) and AB=1 (abnormal). The various combinations available, result in four possible service request/serial poll sequences:

- a) All service request qualifiers are masked (disabled):  
The first serial poll will show the error flags EF3-EF0 for the abnormal condition (if one exists). At the following serial poll, the AB bit will be cleared and the 'normal' status is indicated by EF3-EF0.
- b) One or more AB=1 conditions are enabled. All AB=0 conditions are masked (disabled): If the enabled Abnormal condition qualifier occurs, a service request is generated and the abnormal condition is indicated in the status byte.
- c) All AB=1 conditions are masked (disabled). One or more AB=0 conditions are enabled: If the enabled 'normal' condition qualifier occurs, a service request is generated. The first serial poll will show the error flags EF3-EF0 for the 'normal' condition that caused the service request. A following serial poll will set AB to 1 and indicate the 'abnormal' status.
- d) Some or all 'normal' and 'abnormal' qualifying conditions are enabled: If the enabled 'abnormal' condition occurs, a service request is generated and the reason is indicated in the status byte. If the enabled 'normal' condition occurs a service request will be generated and the reason is indicated in the status byte. However, if an 'abnormal' condition occurs before the pending 'normal' request is polled, the 'abnormal' condition will be indicated in the status byte (the reason for the 'normal' condition is lost!!!!).